Application Note

S32K MAC 应用说明

S32K3 平台系列

AN01010101 1.2.00 Date:2025/12/24

类别	内容
关键词	EMAC 模块 应用笔记
摘要	S32K344

修订历史

版本	日期	原因
V1.0.00	2021/7/2	创建文档
V2.0.00	2022/12/22	更新 RTD2.0.0
V4.0.00	2023/12/15	更新 RTD4.0.0
V6.0.00	2025/11/16	更新 RTD6.0.0

目 录

1. S32K3-EMAC 概述.....1
1.1 常见问题.....2
2. S32K3-EMAC-RTD 配置说明.....3
2.1 版本说明.....3
2.2 EMAC 配置详解.....3
3. 例程解读.....7
3.1 例程解读-S32K344_GMAC_RTD600.....7
4. 免责声明.....12

1. S32K3-EMAC 概述

S32K3xx 的 MAC IP 除了 IEEE802.3 规范中定义的默认功能之外，还支持这些功能：

- 支持 MII/RMII 接口
- 支持符合 IEEE802.3-2015 规范的 10/100 Mbps 应用程序
- 以太网 MII 接口支持 200Mbps 数据速率
- 时间感知整形器（IEEE802.1BV），时间同步（IEEE802.1AS-REV），以及用于时间敏感网络的帧抢占（IEEE802.1QBU）
- 媒体时钟恢复和 AVB 的生成
- ABA 2.0 对于 AHB 主站和 APB3 接口
- RMII 规范版本 1.2 来自 RMII Consortium
- 用于传输，接收和控制路径的单独接口
- 双缓冲
- 广播/多播数据包复制
- 全双工流量控制操作（IEEE 802.3x 暂停数据包和优先流量控制）
- 使用 MAC 管理（RMON）计数器网络统计数据
- 灵活地控制每秒脉冲（PPS）输出信号
- 支持 MDIO
- 序言和 SFD 插入和删除
- 自动 CRC 和垫片生成/剥离选项
- 禁用 CRC 检查的选项
- 可编程插入包间隙
- 源地址插入或替换
- VLAN 插入，替换和删除在传输的数据包中用每包或静态全局控制，插入，替换和删除最多两个 VLAN 标签，插入，替换和删除的队列或基于频道的 VLAN 标签
- IEEE802.1Q VLAN 标签检测和选项可在接收数据包中删除
- 可编程安全监视器超时限制
- 灵活的地址过滤模式，允许：
  - 最多两台额外的 48 位完美 DA 过滤器，每个字节都有掩码
  - 最多两个 48 位 SA 比较检查每个字节的掩码
  - 用于多播和单播 DA 地址的 64 位哈希滤波器
  - 选项可通过所有多传电件的地址数据包
  - 混杂模式通过所有数据包，无需任何过滤网络监控
  - 使用状态报告的所有传入数据包（根据过滤器）
- 附加的数据包过滤，即基于：
  - VLAN 标签 - 完美匹配和基于哈希的过滤，基于外部或内部 VLAN 标签，是可以的

- 第 3 层和第 4 层 TCP 或 UDP OVER IPv4 或 IPv6
- 基于四个过滤器选择的扩展 VLAN 标签过滤

1.1 常见问题

1. 配置 PHY 产生时钟给 S32K3 的 MAC 时，需要 PHY 正常工作，能提供正确的时钟后，才可以对 MAC 的时钟进行初始化，否则配置 MAC 的时钟源可能是错误的，导致数据发送不正确。

2. S32K3-EMAC-RTD 配置说明

2.1 版本说明

S32DS 平台为 S32DS 3.6.3

本次配套例程所使用的 SDK 包版本为 6.0.0，具体 RTD 包为 SW32K3_S32M27x_RTD_R21-11_6.0.0_D2506_DesignStudio_updatesite 和 SW32K3_S32M27x_RTD_R21-11_6.0.0_QLP04_HF01_D2510_DesignStudio_updatesite。

注：提供的例程不能使用 S32DS 3.5 或者之前的版本，否则打开工程编译可能会报错。

2.2 EMAC 配置详解

双击如图 2.1 所示红色部分，进入配置界面。

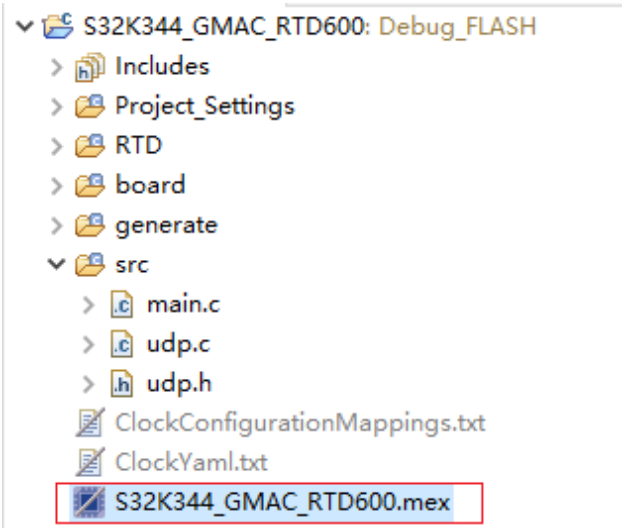


图 2.1 进入配置界面

如图 2.2 所示，将 EMAC 配置 MII 接口模式，并且配置好 MDIO 和 PHY 的复位以及 LED 引脚。

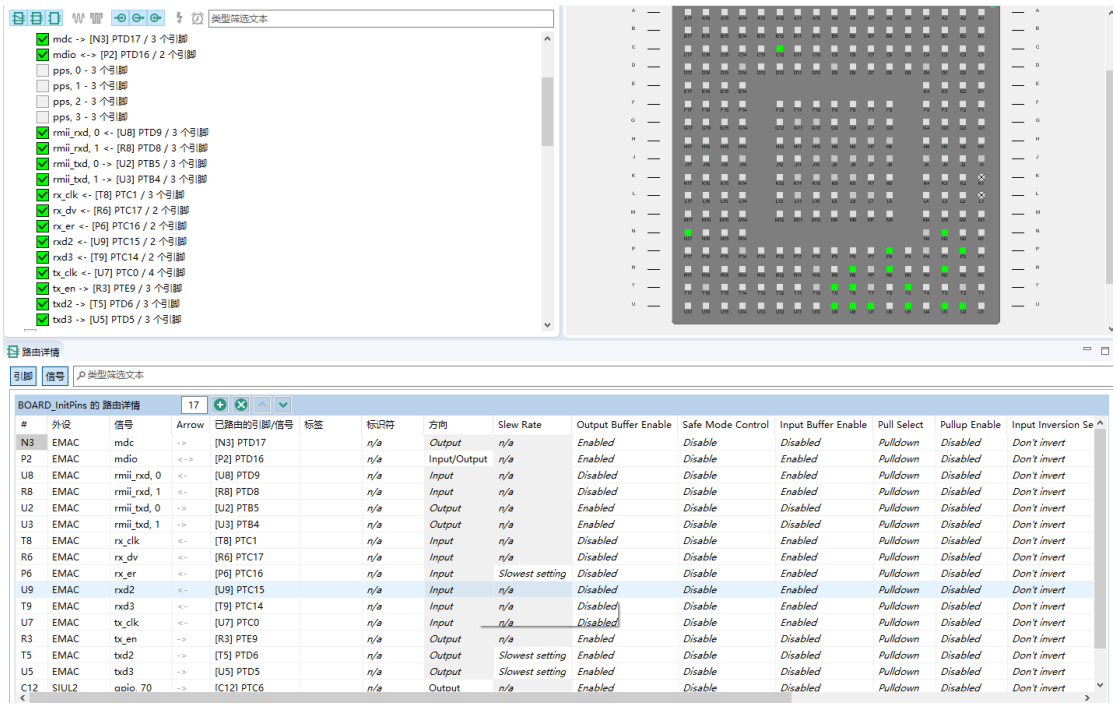


图 2.2　引脚配置

如图 2.3 所示，EMAC 的时钟有三个，分别是：RX_CLK、TS_CLK、TX_CLK。TS_CLK 是模块的时钟，直接选择 PLL 时钟即可。RX 和 TX 的时钟根据所选的 xMII 接口进行配置。

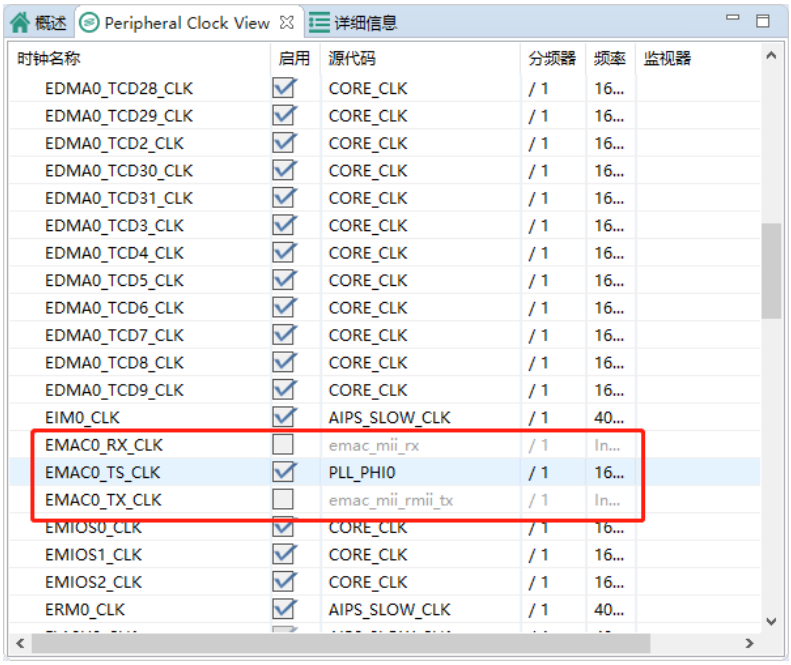


图 2.3　LPUART 时钟配置

如图 2.4 所示，添加工程必要的驱动 IP。

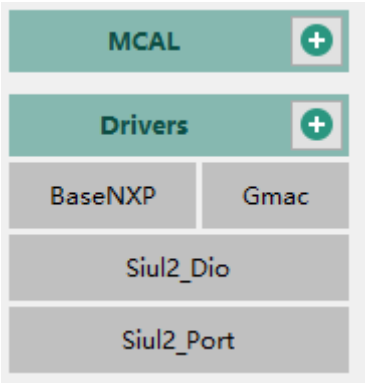


图 2.4　添加驱动 IP

如图 2.5、图 2.6、图 2.7 所示，对 EMAC 进行简单的配置，实现 EMAC 的基本收发配置，其他的 EMAC 配置项使用默认值即可，勾选了 TX 和 RX_Buffer 的分配

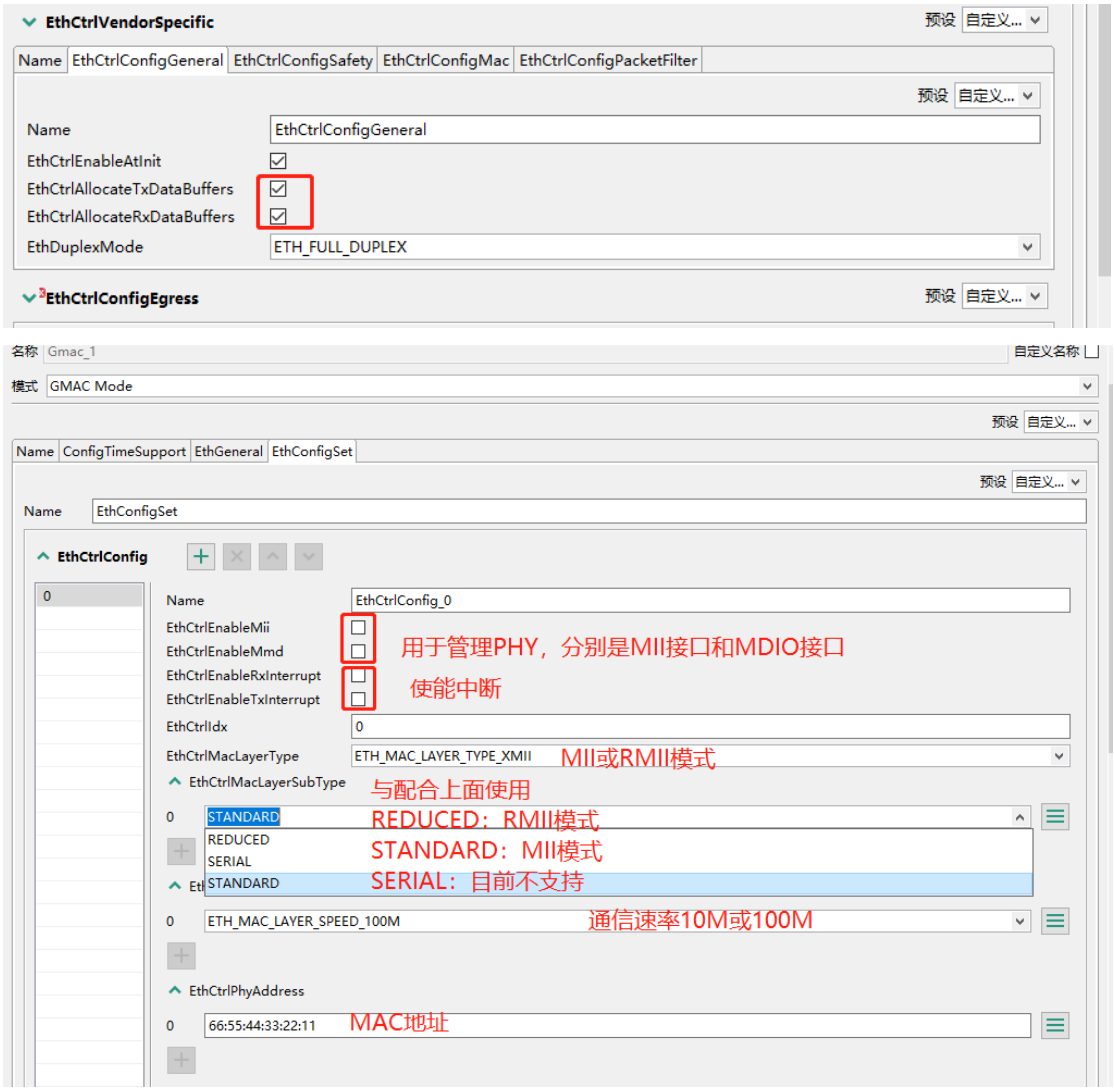


图 2.5　EMAC 配置 1

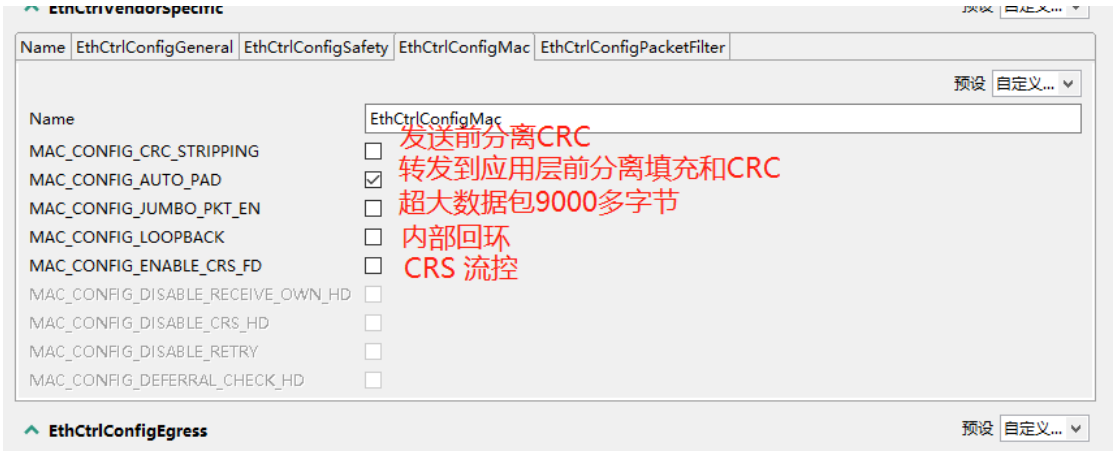


图 2.6　EMAC 配置 2

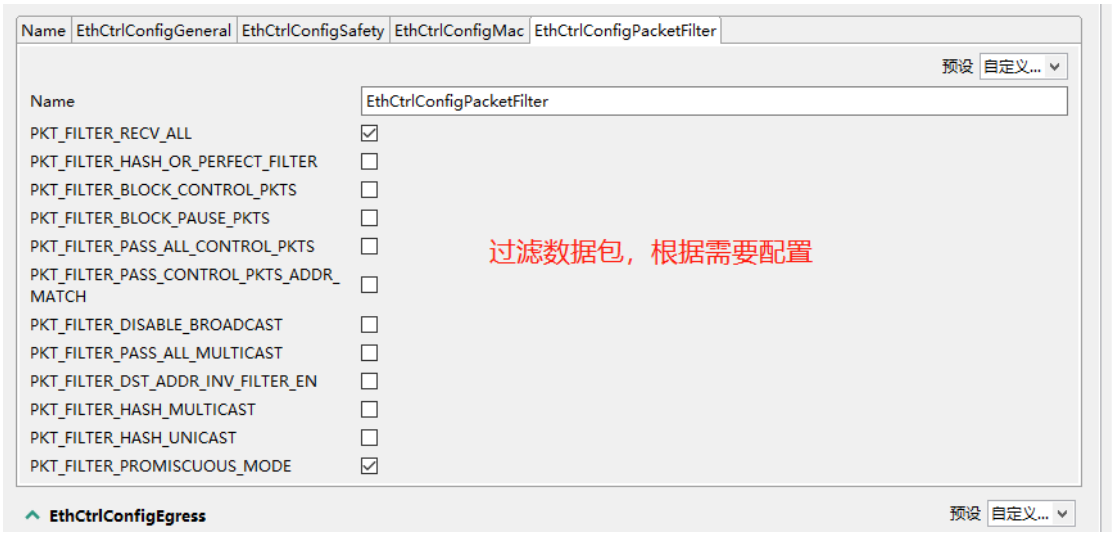


图 2.7　EMAC 配置 3

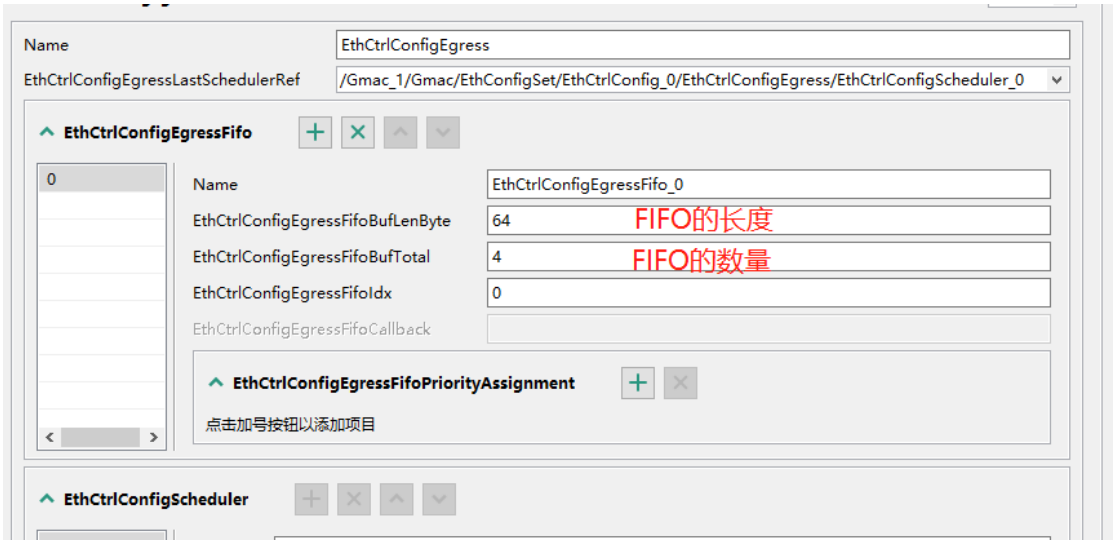


图 2.8　EMAC 配置 4

3. 例程解读

3.1 例程解读-S32K344_GMAC_RTD600

如程序清单 3.1S32K344 只有一个 EMAC，所以 MAC 和 SMI 的接口都是 0，PHY 的地址默认配置为 2；例程采用自发自收的模式，即不需要通过外接板子或上位机通信，展示以太网通信的效果。

详细说明，请参照例程注释。

程序清单 3.1 EMAC 通信例程

```
int main(void)
{
    Gmac_Ip_TxOptionsType TxOptions = {TRUE, GMAC_CRC_AND_PAD_INSERTION,
GMAC_CHECKSUM_INSERTION_DISABLE};
    Gmac_Ip_BufferType TxBuffer = {0};
    Gmac_Ip_TxInfoType TxInfo   = {0};
    Gmac_Ip_BufferType RxBuffer = {0};
    Gmac_Ip_RxInfoType RxInfo   = {0};
    boolean Result = TRUE;
    Gmac_Ip_StatusType Status;
    uint8 MacAddr[6U] = {0U};
    uint8 DstAddr[6] = {0x22,0x33,0x44,0x55,0x66,0x99};
    uint8 i;
    volatile uint8 tmp[64] = {0};

     uint16_t reg0;
     uint16_t reg1;
     uint16_t reg17;
     uint16_t reg18;
     uint16_t reg19;
     uint16_t reg21;
     uint16_t reg20;
     uint16_t reg23;
     uint16_t reg25;
     uint16_t reg24;
     uint16_t reg26;

#if UDP_T
    uint8_t data_tx[8] = {0x61, 0x62, 0x63, 0x64, 0x65, 0x66, 0x67, 0x68};
#endif

    /*  系统初始化  */
    OsIf_Init(NULL_PTR);

    /* GPIO 初始化  */
```

```
    Siul2_Port_Ip_Init(NUM_OF_CONFIGURED_PINS_PortContainer_0_VS_0,
g_pin_mux_InitConfigArr_PortContainer_0_VS_0);

    volatile uint32 timeout = 0xFFFF;
    while(timeout--){}
    /* 时钟初始化 */
    Clock_Ip_Init(&Clock_Ip_aClockConfig[0U]);

    /* GMAC 初始化 */
    Gmac_Ip_Init(INST_GMAC_0, &Gmac_0_ConfigPB);

    /* 使能 SMI 读写 PHY 寄存器*/
    Gmac_Ip_EnableMDIO(0, 0, 100000000UL);

    /* 得到主机 MAC 地址 */
    Gmac_Ip_GetMacAddr(INST_GMAC_0, MacAddr);

     Gmac_Ip_MDIOWrite(0,0,17,0x9804,1000);

     Gmac_Ip_MDIOWrite(0,0,18,0xDC01,1000);/*主机*/
     Gmac_Ip_MDIOWrite(0,0,0,0x6100,1000);

     Gmac_Ip_MDIOWrite(0,0,17,0x9800,1000);

    /* 申请一个至少 64 字节的 Buffer */
    TxBuffer.Length = 64U;
    /* 关联 TxBuff 和 MDA 通道 */
    if ((GMAC_STATUS_SUCCESS != Gmac_Ip_GetTxBuff(INST_GMAC_0, 0U, &TxBuffer, NULL_PTR)) ||
(TxBuffer.Length < 64U))
    {
        Result = FALSE;
    }

    /* 把不同的帧放入 TxBuffer */
#if Ethernet

    /* 做一个以太网帧 */
    mac_frame frame;
    for (i = 0U; i < 6U; i++)
    {
     frame.srcMacaddress[i] = MacAddr[i];
     frame.destMacaddress[i] = DstAddr[i];
    }
    frame.length = 60;
```

```
    /* 数据长度 = Frame - (DstAddr + SrcAddr + EtherType/Length + FCS) 64U - (6U + 6U + 2U + 4U)*/
    for(i = 0; i < 46; i++)
      {
          frame.data[i] = i;
      }

    /* 把帧放入 Buffer */
    copy_buffer(*(&TxBuffer.Data),(uint8_t *)&frame,60);
    for(i = 0; i < 64; i++)
    {
     tmp[i] = *((uint8 *)(&TxBuffer.Data[0U] + i));
    }
#endif
     /* 调试查看 PHY 是 否异常*/
     Gmac_Ip_MDIORead(0,0,0,&reg0,1000); //8448   全字节 100M
     Gmac_Ip_MDIORead(0,0,1,&reg1,1000); //485 link_up 自动协商完成
     Gmac_Ip_MDIORead(0,0,17,&reg17,1000);//38912   接上且正常模式
     Gmac_Ip_MDIORead(0,0,18,&reg18,1000);//56325   56321
     Gmac_Ip_MDIORead(0,0,19,&reg19,1000);//4677    5189
     Gmac_Ip_MDIORead(0,0,21,&reg21,1000);//
     Gmac_Ip_MDIORead(0,0,20,&reg20,1000);//
     Gmac_Ip_MDIORead(0,0,23,&reg23,1000);//48356
     Gmac_Ip_MDIORead(0,0,24,&reg24,1000);//0                    16384
     Gmac_Ip_MDIORead(0,0,25,&reg25,1000);//0
     Gmac_Ip_MDIORead(0,0,26,&reg26,1000);//0

#if UDP_T
    /* 做一个 UDP 包*/
     buid_udp_packet(TxBuffer.Data, data_tx, 8);
#endif

    while(1)
    {
     do
     {
          Gmac_Ip_MDIORead(0,0,1,&reg1,1000); //485 link_up 自动协商完成
     }while((reg1 & 0x4) != 0x4);
#if SEND
     /* 发送数据 */
          TxBuffer.Length = 64U - 4U;       /* 不计算 FCS，因为这 4 字节会被控制器自动插入 */
          if (GMAC_STATUS_SUCCESS != Gmac_Ip_SendFrame(INST_GMAC_0, 0U, &TxBuffer,
&TxOptions))
          {
```

```
            Result = FALSE;
        }

        /* 等待发送完成*/
        do {
            Status = Gmac_Ip_GetTransmitStatus(INST_GMAC_0, 0U, &TxBuffer, &TxInfo);
        } while (Status == GMAC_STATUS_BUSY);

        /* 判断发送状态  */
        if ((GMAC_STATUS_SUCCESS != Status) || (0U != TxInfo.ErrMask))
        {
            Result = FALSE;
        }
#endif

#if RECV
        /* 接收数据 */
        do{
            Status = Gmac_Ip_ReadFrame(INST_GMAC_0, 0U, &RxBuffer, &RxInfo);
        }while(Status == GMAC_STATUS_RX_QUEUE_EMPTY);
        /* 提供一个接收数据的 buffer */
         Gmac_Ip_ProvideRxBuff(INST_GMAC_0, 0U, &RxBuffer);
         /* debug 查看接收数据*/
        for (i = 0U; i < 60; i++)
        {
            if( tmp[i] != *((uint8 *)(&RxBuffer.Data[0U] + i)))
            {
                Result = FALSE;
                break;
            }
        }

#endif
        if(Result == TRUE)
        {
        /* 成功发送了 数据 */
        Siul2_Dio_Ip_ClearPins(LED4_PORT,(1 << LED4_PIN));
        ;
        }
        /* 调试查看 PHY 是 否异常*/
        Gmac_Ip_MDIORead(0,0,0,&reg0,1000); //8448  全字节 100M
        Gmac_Ip_MDIORead(0,0,1,&reg1,1000); //485 link_up 自动协商完成
        Gmac_Ip_MDIORead(0,0,17,&reg17,1000);//38912  接上且正常模式
        Gmac_Ip_MDIORead(0,0,18,&reg18,1000);//56325  56321
```

```
            Gmac_Ip_MDIORead(0,0,19,&reg19,1000);//4677      5189
            Gmac_Ip_MDIORead(0,0,21,&reg21,1000);//
            Gmac_Ip_MDIORead(0,0,20,&reg20,1000);//
            Gmac_Ip_MDIORead(0,0,23,&reg23,1000);//48356
            Gmac_Ip_MDIORead(0,0,24,&reg24,1000);//0                    16384
            Gmac_Ip_MDIORead(0,0,25,&reg25,1000);//0
            Gmac_Ip_MDIORead(0,0,26,&reg26,1000);//0
    }
    /* 失能控制器 */
    Gmac_Ip_DisableController(INST_GMAC_0);

    return 0;
}
```

4. 免责声明

本着为用户提供更好服务的原则，广州立功科技股份有限公司（下称“立功科技”）在本手册中将尽可能地为用户呈现详实、准确的产品信息。但介于本手册的内容具有一定的时效性，立功科技不能完全保证该文档在任何时段的时效性与适用性。立功科技有权在没有通知的情况下对本手册上的内容进行更新，恕不另行通知。为了得到最新版本的信息，请尊敬的用户定时访问立功科技官方网站或者与立功科技工作人员联系。感谢您的包容与支持！

专业 . 专注成就梦想

Dreams come true with professionalism and dedication.